

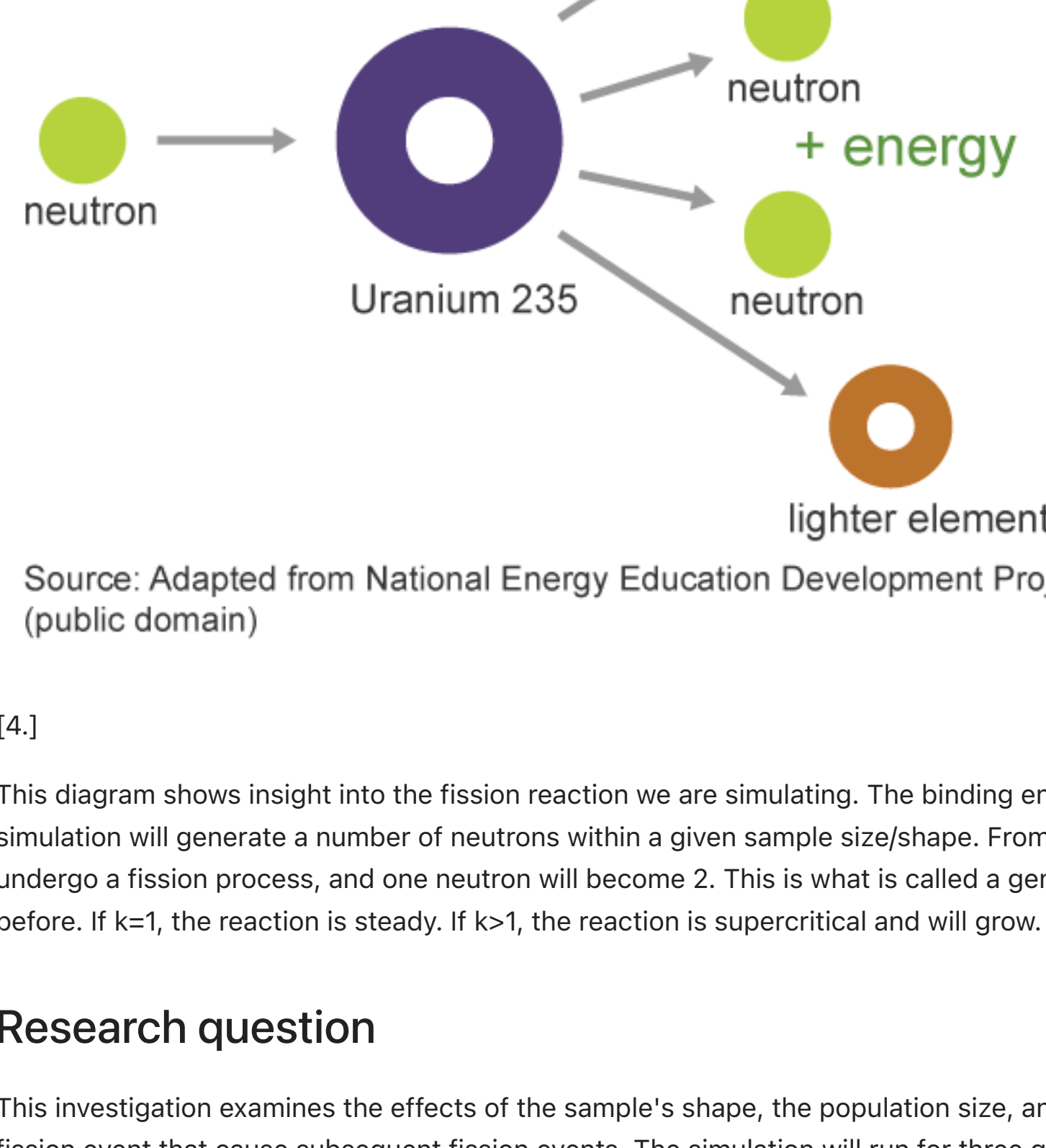
Project 03: Fission process of unstable isotope Uranium 235

Name: Yasmimah Hassan

Introduction

Enrico Fermi was a renowned physicist who passed away in 1954 at the age of 51. During the 1940's, he worked at the University of Chicago, where he turned his attention to high-energy physics and led groundbreaking investigations into pion-nucleon interactions. Fermi also designed and built the first nuclear reactor, leading the epochal experiment that demonstrated the first self-sustained nuclear chain reaction. This investigation will simulate the first-ever self-sustaining artificial nuclear reaction using Uranium-235. [2]

How fission splits the uranium atom



Source: Adapted from National Energy Education Development Project (public domain)

[4]

This diagram shows insight into the fission reaction we are simulating. The binding energy that was used to keep them together turns into kinetic energy and sends them moving in any direction. The simulation will generate a number of neutrons within a given sample size/shape. From there, the neutrons will travel a certain distance based on an average. If they are still found within the sample, they undergo a fission process, and one neutron will become 2. This is what is called a generation. The multiplication factor, K , is a ratio of the number of neutrons after a generation to the number of neutrons before. If $k=1$, the reaction is steady, if $k>1$, the reaction is supercritical and will grow. Lastly, if $k<1$, the reaction is subcritical and will die out. [1]

Research question

This investigation examines the effects of the sample's shape, the population size, and the distance traveled by each neutron on the multiplication factor, K - the average number of neutrons from one fission event that causes subsequent fission events. The simulation will run for three generations, with the third-generation ratio providing sufficient information to draw valid conclusions. Below are the findings of this investigation.

The shape of the same makes unique changes with its surface area and volume ratio. We can expect the sphere to have the highest k value of the 3, with cube next in line. Because of the lack of corners and higher probability of having the most room to travel within a sample. The higher the mean free path is the more likely the neutrons are to leave the same. I expect the number of initial neutrons to be proportionate to the multiplication factor.

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.cm as cm

## function to make a bunch of random initial positions inside a specified shape [2]
def shape(N, shape = 'cube'):
    ## all shapes will use this to begin with, so volume is reasonable similar, but not constant
    a = 0.50 # m
    if shape == 'sphere':
        ## finding theta and phi, like horizontal and vertical angle of all points
        phi = np.random.uniform(0, 2 * np.pi, N)
        cos_theta = np.random.uniform(-1, 1, N)
        theta = np.arccos(cos_theta)
        U = np.random.uniform(0, 1, N)
        r = a * U**(1/3)
        ## multiplying all together to get cartesian coordinates
        x0 = r * np.sin(theta) * np.cos(phi)
        y0 = r * np.sin(theta) * np.sin(phi)
        z0 = r * np.cos(theta)

    ## do the same for pyramid shape
    elif shape == 'pyramid':
        U = np.random.uniform(0, 1, N)
        z0 = a * (1 - U**(1/3))
        bound = a - z0 # x, y coordinates are bound by height, see appendix 1.3 for a sketch
        x0 = np.random.uniform(-bound, bound, N)
        y0 = np.random.uniform(-bound, bound, N)
        ## cube is simple
    elif shape == 'cube':
        x0 = np.random.uniform(0, a, N)
        y0 = np.random.uniform(0, a, N)
        z0 = np.random.uniform(0, a, N)

    ## put em all together and return initial positions in designated shape
    pos0 = np.stack([x0, y0, z0], axis=1)
    return pos0

## monte carlo func,
def U235(N, pos, mean_free_path = .15, shape='cube'):# in m
    a = 0.50 # m
    # Fermi's best guess of the mean free path, = 0.15 m
    # based on the neutron cross section in U235
    d = np.random.exponential(mean_free_path, N)

    # Generate random travel directions
    ##Randomly generate the directions of travel, in three dimensions, of the N0 initial thermal neutrons
    ## using same method as in shapes
    phi = np.random.uniform(0, 2 * np.pi, N)
    theta = np.random.uniform(0, 2*np.pi, N) # other angle
    directions = np.stack((np.sin(theta) * np.cos(phi),
                           np.sin(theta) * np.sin(phi),
                           np.cos(theta)), axis=1)

    # Compute final positions
    final_pos = pos + (directions * d), None)
    # None is necessary to make the array, just expanding the check to iterate over all points given
    # the same size to be multiplied correctly

    # Check which neutrons are captured (remain in the region)
    ## this will be different for all shapes, so different if statements are required
    ## uses the same logic as in the shape func, just expanding the check to iterate over all points given
    if shape == 'sphere':
        inside = np.sqrt(np.sum((final_pos**2)) <= a)
    elif shape == 'pyramid':
        bound = a-final_pos[:,2]
        inside = np.all((0 <= final_pos[:,2]<=a)\
                        & (abs(final_pos[:,0]) <= bound)\
                        & (abs(final_pos[:,1]) <= bound) , axis=1)
    elif shape == 'cube':
        inside = np.all((0 <= final_pos) & (final_pos < a), axis=1)

    ## this is to hold all the new positions of the new neutrons, basically just adding an array to itself,
    ## duplicate it and add it to itself, this way we can have each neutron have its own independant direction next gen
    # [3].## (axis = 0 just means in adding it row wise, instead of column
    v=np.concatenate((final_pos[inside], final_pos[inside]), axis=0)

    ## each neutron still left in the region goes through fission, so double the number of neutrons
    N1 = 2*np.sum(inside)

    ## brief if statement to keep from getting a divide by 0 error
    ## k is ratio of neutrons after first gen fission to initial number of neutrons
    if N != 0:
        k = N1/N
    else:
        k = 0
    ## split back number of neutrons left after 1 gen, k val, and the array of the positions of all the particles
    return N1, k, v

# Neutrons that are still inside the sample after traveling their distance L are considered to have been captured
# and will initiate a new fission event where they were captured. Neutrons that do not remain inside the sample after
# traveling their distance L are lost and will not initiate any further fission events.

##### Investigating the phase space #####
# choosing some value ranges to investigate, initial number of neutrons and the mean free path
N = np.linspace(100, 400, 10)
landuh = np.linspace(.010, .5, 10)

## meshgriding to find all possible combos of params
N, landuh = np.meshgrid(N, landuh)
params = np.column_stack((N.ravel(), landuh.ravel()))

##initializing the params array, for plotting,
Params = np.zeros((params.shape[0], 5))

## setting first 2 columns to be all the params we just found above, to make it all into 1 array for simplicity
# (row start/row end, column start/column end)
Params[:, :2] = params

## mini function to iterate over all 3 gens and return just the k3, cuz thats all we need for plotting
def l_jen():
    k3N, landuh, shapes =
    def pos0 = shape(int(N), shapes)
    N1, k1, final1 = U235(int(N), pos0, landuh)
    N2, k2, final2 = U235(N1, final1, landuh)
    N3, k3, final3 = U235(N2, final2, landuh)
    return k3

## send each shape into function and then update params array with answer, 3rd column = sphere, 4th = pyramid and, last cube
for i, shapes in enumerate(["sphere", "pyramid", "cube"]):
    Params[:, 2 + i] = np.array([k3N, landuh, shapes] for N, landuh in Params[:, :2]))

##### Plotting #####
Fig, Ax = plt.subplots(1, 3, figsize = (20, 10), sharex=True, sharey=True)

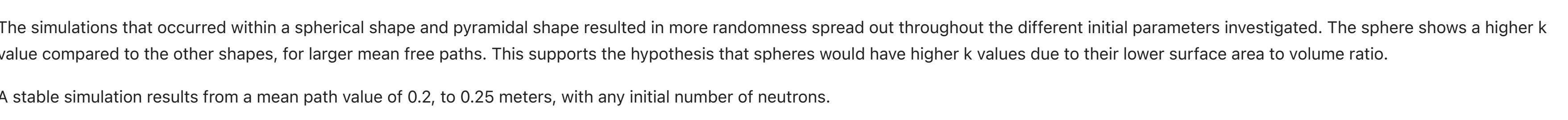
Ax[0].scatter(Params[:,0], Params[:,1], c = Params[:,2], s = 100)
Ax[0].set_title("Sphere", size = 18)
Ax[0].set_ylabel("Mean Free Path", size = 16)

Ax[1].scatter(Params[:,0], Params[:,1], c = Params[:,2], s = 100)
Ax[1].set_title("Pyramid", size = 18)
Ax[1].set_ylabel("Initial Number of Neutrons", size = 16)

Ax[2].scatter(Params[:,0], Params[:,1], c = Params[:,2], s = 100)
Ax[2].set_title("Cube", size = 18)

norm = plt.Normalize(vmin=np.min(Params[:, 2:]), vmax=np.max(Params[:, 2:]))
cmap = cm.viridis
cbr = Fig.colorbar(cm.ScalarMappable(norm = norm, cmap=cmap), ax=Ax, location = 'bottom')
cbr.set_label("Multiplication Factor of Third Generation", size = 16)

plt.show()
```



Conclusion

The plots above represent several simulations run for a given shape, each with a personalized initial number of neutrons and mean free path. This was carried over for all three shapes to derive a contrast between them, as well as between the initial parameters.

The colour of the dots represent the K value after 3 generations, 3 generations give it ample time to settle in. A higher value of k , and a lighter colour indicates a supercritical generation, while a darker colour, a lower k , represents a subcritical reaction. we can see a trend of decreasing k values as mean free path increases.

[8]

We can see that the cube graph has the best and clearest gradient, therefore it is the most consistent with the initial number of particles and mean free path. The relationship between the initial number of neutrons and the mean free path is more constant than I was expecting, upon further investigation. This accurately represents the simulation. Beginning with more neutrons will not necessarily impact the number of fission reactions to occur, it just means a higher number of neutrons will exit the system.

The free mean path however does have a dominant affect on the simulation. As the average length of travel increases, the resulting neutrons still left within the region will lessen. Most neutrons will travel far out of the enclosure region.

The simulations that occurred within a spherical shape and pyramidal shape resulted in more randomness spread out throughout the different initial parameters investigated. The sphere shows a higher k value compared to the other shapes, for larger mean free paths. This supports the hypothesis that spheres would have higher k values due to their lower surface area to volume ratio.

A stable simulation results from a mean path value of 0.2, to 0.25 meters, with any initial number of neutrons.

A limitation of this simulation is the the phase space itself. The number of neutrons had no identifiable impact on the simulation as I expected and an investigation looking into another parameter could hold some more weight in the investigation.

References

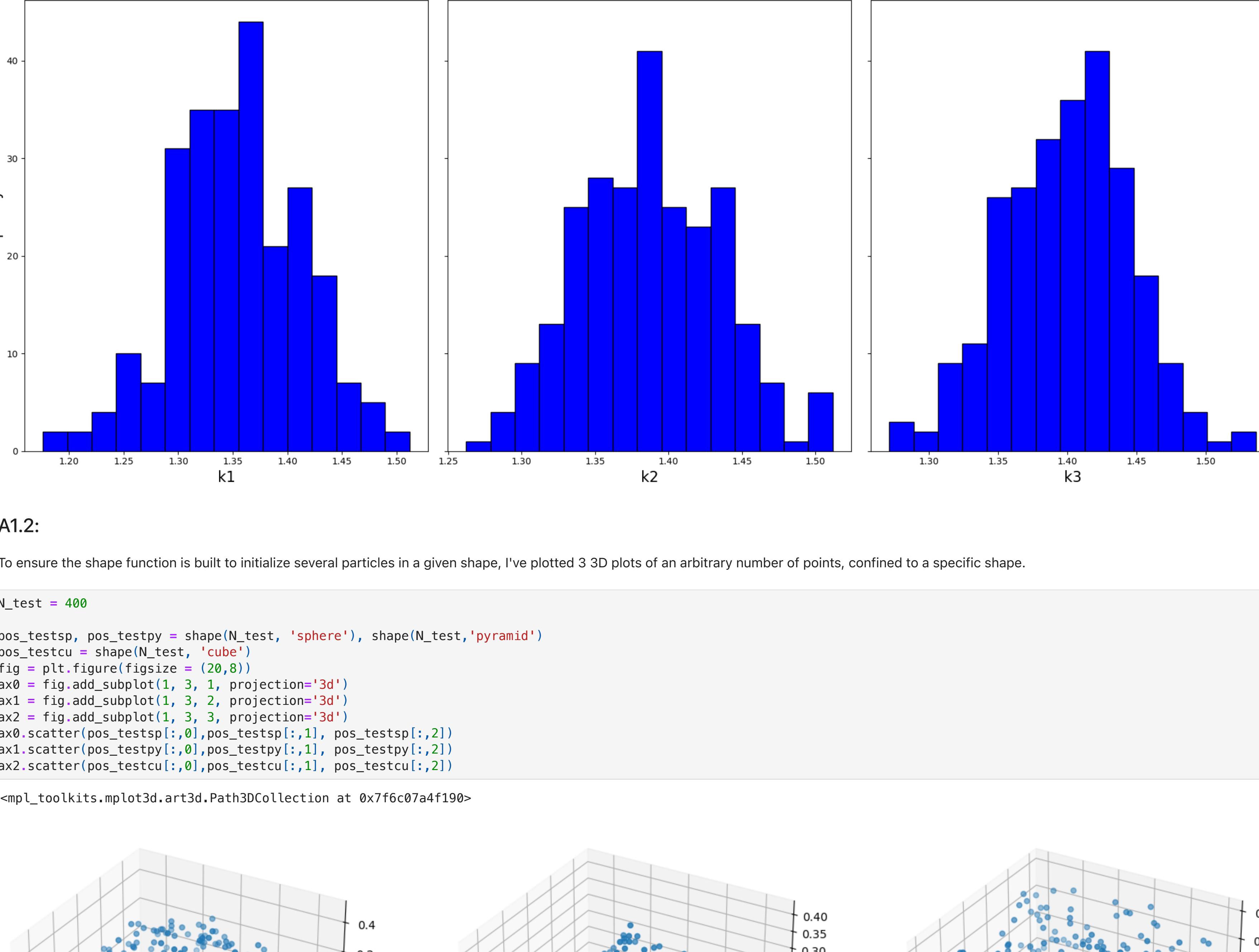
- ChatGPT
- Efficiency in code
- <https://www.nobelprize.org/prizes/physics/1938/fermi/biographical/>
- Enrico Fermi's Bibliography
- <https://www.geeksforgeeks.org/numpy-concatenate-function-python/>
- Concatenate 2 arrays
- <https://www.eia.gov/energyexplained/nuclear/#~:text=All%20nuclear%20power%20plants%20use,when%20a%20uranium%20atom%20splits.>

Uranium fission diagram

Appendix 1: Code validation

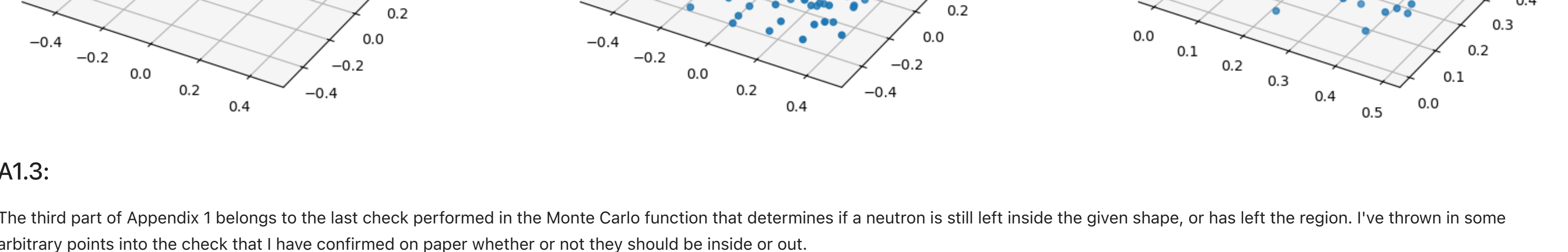
A1.1: Comparison of simulation results with by-hand calculations

For the first part for the Appendix, I want to plot the Histograms of the 3 generations to ensure the Monte Carlo simulation I've built, accurately calculates the multiplication factor. We expect to see a increase in the average K values over generations, and a decrease of standard deviation.



A1.2:

To ensure the shape function is built to initialize several particles in a given shape, I've plotted 3 3D plots of an arbitrary number of points, confined to a specific shape.



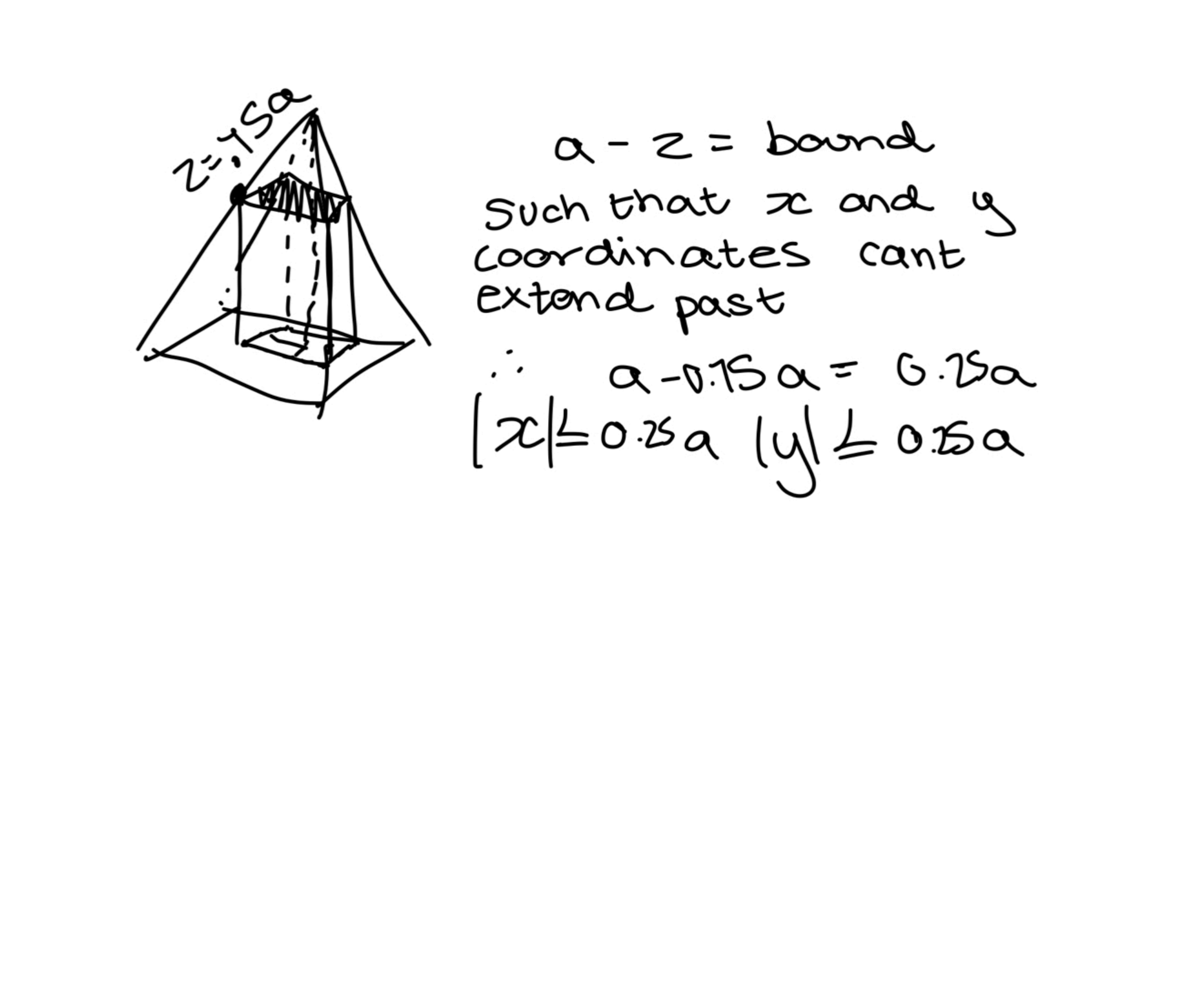
A1.3:

The third part of Appendix 1 belongs to the last check performed in the Monte Carlo function that determines if a neutron is still left inside the given shape, or has left the region. I've thrown in some arbitrary points into the check that I have confirmed on paper whether or not they should be inside or out.

```
in [18]: a= .5
        final_pos_testcu = [[2*a,0,0]]## should be outside cube
        final_pos_testpy = [(-.25*a,.25*a,.75*a)]## should be inside pyramid
        final_pos_testsp = [(a,a,0)]## should be outside sphere
        check_test = ['cube', final_pos_testcu, 'pyramid', final_pos_testpy, 'sphere', final_pos_testsp]

        for row in check_test:
            shape = row[0]
            final_pos = np.array(row[1])
            if shape == 'sphere':
                inside = np.sqrt(np.sum((final_pos**2)) < a)
                if inside == True:
                    print(final_pos, "is inside the sphere")
                else:
                    print(final_pos, "is not inside the sphere")
            elif shape == 'pyramid':
                bound = a-final_pos[2]
                inside = np.all((0 <= final_pos[2]<=a)\
                                & (abs(final_pos[0]) <= bound)\
                                & (abs(final_pos[1]) <= bound) )
                if inside == True:
                    print(final_pos, "is inside the pyramid")
                else:
                    print(final_pos, "is not inside the pyramid")
            elif shape == 'cube':
                inside = np.all((0 <= final_pos) & (final_pos < a))
                if inside == True:
                    print(final_pos, "is inside the cube")
                else:
                    print(final_pos, "is not inside the cube")

[1, 0, 0.] is not inside the cube
[-0.125  0.125  0.375] is inside the pyramid
[0.5 0.5 0.] is not inside the sphere
```



Appendix 2: Reflection questions

Reflection 1: Coding Approaches (A)

(How well did you apply and extend your coding knowledge in this project? Consider steps you took to make the code more efficient, readable and/or concise. Discuss any new-to-you coding techniques, functions or python packages that you learned how to use. Reflect on any unforeseen coding challenges you faced in completing this project.)

The histogram plots were relatively new to me, so I reviewed the lecture slides from the 16th, to further extend the plots to make them look good, and effectively convey information.

Reflection 2: Coding Approaches (B)

(Highlight an aspect of your code that you feel you did particularly well. Discuss an aspect of your code that would benefit the most from further effort.)

I worked really hard to develop the "shape" function, I was plotting the 3D plot along with the development of the code to help with debugging. I found it difficult to determine cartesian coordinates inside a pyramid because the x and y coordinates are dependent on the z.

Reflection 3: Simulation physics and investigation (A)

(How well did you apply and extend your physical modelling and scientific investigation skills in this project? Consider the phase space you chose to explore and how thoroughly you explored it. Consider how you translated physics into code and if appropriate any new physics you learned or developed a more thorough understanding of.)

I chose to investigate the dependence of the initial number of neutrons and the average distance traveled, through different geometric shapes. This meant I had multiple relationships to comment on. I explored these relationships quite thoroughly and came to concise conclusions of my findings

Reflection 4: Simulation physics and investigation (B)

(Highlight something you feel you did particularly well in terms of the context of your simulation, the physical modelling that you did or the investigation you performed. Discuss an aspect of these dimensions of your project that would benefit the most from further effort.)

While the phase space I chose to investigate did result in a conclusion, I am not yet satisfied with the phase space. I think a different phase space would have offered insight into the simulation that I would not have expected otherwise.

Reflection 5: Effectiveness of your communication

(Highlight something you feel you did particularly well in your visualizations or written communication. Discuss an aspect of your visualizations or written communication that would benefit the most from further effort.)

Appendix 1, is filled with the most valuable and convincing material for the simulation. While building the code, I checked for errors and then fixed them, and showed how I accomplished that.

In [23]: